

California Housing — Area Valuation Model

Decision Support Report | Yunus Emre Capar | March 2026 | Track A – Regression

1. Goal & Selected Track

A consulting client working with municipalities and real estate companies needs a tool to estimate median house values for California districts. They want to screen areas quickly — without manual appraisals — to support investment planning and resource decisions.

Selected track: Track A (Regression) — the client needs an actual price estimate, not just a yes/no answer.
Target variable: *median_house_value*

2. Data Insights (EDA)

The dataset has 20,640 rows and 10 columns describing California census districts. Three things stood out:

- **Income drives price.** *median_income* has the strongest correlation with house value. Higher-income areas consistently cost more.
- **The target is capped.** A large number of records hit a ceiling at \$500,001. This limits what the model can learn about high-value areas.
- **Location matters.** Ocean proximity clearly separates price levels — ISLAND and NEAR BAY areas are significantly more expensive than INLAND.
- **Raw totals are less useful than ratios.** Features like *total_rooms* scale with district size. Ratio features (rooms per household) carry more real signal.

3. Method

Why a test set? The data was split 80/20 — 16,512 rows for training, 4,128 held back for testing. The test set is locked away until the very end. This simulates real deployment: the model gets judged on data it has never seen, so the results are honest.

Feature engineering. Three ratio features were created: *rooms_per_household*, *bedrooms_per_room*, and *population_per_household*. These turned out to be more useful than the raw totals.

Preprocessing pipeline. A scikit-learn Pipeline handled all preprocessing inside cross-validation — median imputation for missing values, *StandardScaler* for numerical features, and *OneHotEncoder* for *ocean_proximity*. Doing it inside the pipeline ensures test data never influences training.

Models tested: Baseline (*DummyRegressor*), Linear Regression, Random Forest, Gradient Boosting.

Evaluation metric: MAE. It tells you in plain dollars how far off predictions are on average — easy to explain to non-technical stakeholders. 5-fold cross-validation was used for a stable comparison.

4. Results

Model	MAE (cross-val)	R ²
Baseline (<i>DummyRegressor</i>)	\$91,391	0.000
Linear Regression	\$48,880	0.655
Gradient Boosting	\$36,608	0.791
Random Forest	\$32,911	0.811

After hyperparameter tuning with *GridSearchCV* (108 model fits), the best settings were: 300 trees, unlimited depth, sqrt max features. The improvement from tuning was modest — the default settings were already close to optimal.

Final performance on unseen test data:

MAE	RMSE	R ²
\$32,374	\$48,854	0.818

The model is off by \$32,374 on average and explains 81.8% of the variation in house prices.

5. Recommendation

Recommended model: tuned Random Forest. It had the lowest MAE and highest R^2 in both cross-validation and the final test. It was also the most stable — lowest variance across folds ($\pm\$517$ vs $\pm\$1,008$ for the baseline). It is well suited for screening large numbers of districts quickly. It should not be used for precise individual property appraisals.

6. Risks & Limitations

Risk	Details
Price ceiling	The target is capped at \$500,001. Errors for areas above \$400k average \$79,680 vs \$21,169 for areas below \$100k.
ISLAND areas	Only a handful of island districts exist in the data. MAE for ISLAND is \$153,945 — not reliable for these areas.
Old data	The dataset is a historical snapshot. Housing markets change over time. Retraining on current data is needed before real deployment.
District-level only	All features are area aggregates, not individual homes. The model estimates medians, not specific property values.
Outliers	Some engineered features showed extreme values (e.g. population per household = 1,243). These may be data errors.

7. Unsupervised Learning — PCA

What is unsupervised learning? It finds patterns in data without using a target variable. You hand it the raw features and ask: is there hidden structure here?

When is it relevant here? Before or alongside modeling, it helps understand whether districts naturally group together — for example, cheap inland clusters vs expensive coastal ones. This can inform how we segment areas for decision-making.

PCA compresses many correlated features into fewer dimensions while keeping as much information as possible. **KMeans clustering** groups data points into k clusters based on similarity — useful for finding 'types' of districts.

Two risks: (1) PCA components are mathematical abstractions — they lose direct real-world meaning, making communication to non-technical people harder. (2) Both PCA and KMeans are sensitive to outliers and feature scaling — results can change significantly depending on how you preprocess.

Implementation and findings: PCA was applied to all 11 numerical features (after imputation and scaling). 4 components explain 80% of variance. 6 components explain 90%. PC1 (35.5%) is driven by district size features — households, total_bedrooms, total_rooms. PC2 (19.0%) is driven by quality and wealth features — bedrooms_per_room, median_income, rooms_per_household. The 2D projection shows a visible price gradient: cheap areas cluster tightly on the left while expensive areas spread along the quality axis. The data has real structure — it is not random noise.

8. Self-Reflection

What went well: The full pipeline was built cleanly using sklearn Pipelines with no data leakage. Feature engineering added real value — the three ratio features I created ranked in the top 4 most important features. The error analysis gave concrete, actionable findings about where the model works well and where it does not.

What was hardest: Understanding why preprocessing must happen inside the cross-validation loop, not before it. Also translating PCA results into something meaningful beyond just numbers.

Grade assessment: VG. All G requirements are covered — EDA, preprocessing pipeline, 3+ models, cross-validation comparison, hyperparameter tuning, final test evaluation, and report. VG requirements are also met: PCA implementation with interpretation, feature importance analysis, error analysis by price range and category, and critical discussion of risks including data leakage, sparse categories, temporal drift, and generalization limits.